MaxDuino

User guide

INDEX

Acknowledgments

I would like to thank Rafael Molina for his great work on the MaxDuino project, as well as his contributions to improving this document.

License

Version control

Date	Version	Changes
11/04/2020	1.0	Initial version by desUBIKado

1. What are TAPUINOS, TZXDUINOS, CASDUINOS and MAXDUINOS?

This name refers to a series of devices used to load computer programs via audio. It also refers to the firmware these devices use, which can lead to confusion, since it's possible to install TZXduino firmware on a TZXduino device, but also, for example, ToT (Tapuino on Tzxduino), CASduino, or MAXduino firmware.

The files containing the data for audio loading are stored on an SD or microSD card and are of different types depending on the computer they are intended for. For example, there are .TZX files specific to the Sinclair Spectrum, and others like .CDT that are for the Amstrad CPC. Some extensions, such as .CAS or .TAP, can be shared by different computers, but their internal encoding is different, so only the files specific to the computer model for which the audio loading is intended should be used.

The main difference between firmwares is that each one is only capable of playing certain tape files, so some are specialized, although not restricted, to certain computer models, for example, TAPuino with Commodore computers (C64, VIC20, etc.), CASduino with MSX computers, TZXduino with Sinclair computers (Spectrum, ZX80 or ZX81), and MAXduino, which was born as a unification of the TZXduino and CASduino firmwares, has a more generalist approach, and allows loading on a large number of computers and formats.

The Arduino Nano (Atmega328), Arduino Nano Pro, and Arduino Every chips are the ones currently used by the builders of these boards. Recently, the Megaduino project was launched, a board that uses the more powerful Arduino Mega 2560 chip. The current version 1.54 of the maxduino firmware only uses 16% of its capacity, compared to 99% for the other chips, making it a promising option for adding new functionalities, including the ability to record audio, not just play it back like the other boards.

2. What systems does the MaxDuino firmware support?

In its latest version 1.54, the supported systems and file formats are as follows:

- Spectrum: TAP and TZX
- Amstrad CPC: CDT
- MSX: CAS and TSX
- ZX80: O
- ZX81: P
- Acorn Atom and Electron: UEF (uncompressed) and TSX
- Dragon 32/64 / Tandy CoCo: CAS and TSX
- SVI Spectravideo: TSX
- Oric 1 and Oric Atmos: Does not support oric TAPs, but does support TAPs converted to TSX

As you can see, this firmware does not allow loading TAP tape files for Commodore VIC20, C64, C16/C116, or Plus/4. However, if you have a TZXDuino board, you can install the ToT (Tapuino on Tzxduino) firmware, which does allow it, but then you will lose the ability to load files on other systems.

Furthermore, it is possible to incorporate data into TSX files not only from MSX systems, but also from **any computer**that used the encoding*Kansas City Standard.*So on NataliaPC's github (https://github.com/nataliapc/MSX_devs/tree/master/TSXphpclass) we found utilities to convert different types of tape files to TSX:

- **uef2tsx.php**: For the .UEF files of the Atom, Electron and BBC Micro computers from the company Acorn.
- **oric2tsx.php**For Oric .TAP files
- **drg2tsx**For Dragon and Tandy Coco .CAS files
- **svi2tsx**: For SVI Spectravideo .CAS files
- **cas2tsx**: For MSX .CAS files

3. Why use these devices and not a mobile application?

There are mainly 3 advantages:

- These devices read the data directly to be converted into audio without needing to convert it to .wav format beforehand, which takes some time, while playback from the "duinos" is immediate and also consumes storage space. Mobile apps generally don't automatically delete these .wav files after use, and they take up a lot of space if many games are loaded, so in the end you have to delete them manually, which is a nuisance.

- On the other hand, some of these devices (not all) have a remote (REM) connector that allows the computer to control the loading, which is very useful in multi-load games for those computers that had the option to control the datasettes. However, although some models do not have a remote connector, this is not a serious problem because when playback is paused, it is very easy to rewind or fast-forward between blocks simply by using the up or down buttons on these devices.

- Some of the Duo models include amplification, which is very useful for certain computer models that are somewhat sensitive to sound and require a fairly high volume for successful audio loading. This is sometimes difficult to achieve on a mobile phone due to the volume limitations of most devices.

The drawback, obviously, is the price, since mobile apps specifically designed to play these tape files are generally free, and almost everyone has a mobile phone these days. However, the Duos aren't particularly expensive, with prices ranging from €15 to €40 depending on features, whether they include a case, and other factors.

4. What is MaxDuino?

To give a little background, it all started with Sweetlilme and his TAPuino project, focused on loading .TAP audio files onto Commodore machines (PET, VIC20, C64 and C128).

Later, based on this, Duncan Edwards and Andrew Beer created the CASduino to play .CAS files for MSX. Then came TZXduino, created by the same authors, to play .TAP and .TZX files from the Sinclair ZX Spectrum, to which they quickly added .CDT files from the Amstrad CPC (which are basically .TZX files with a different extension).
They continued to improve it and added the ability to play .CAS files from Dragon data and Tandy Color Computers.

Other improvements were also made to the TZXduino, such as support for .AY files (melodies that can be played on the AY-3-8912 sound chips found in computers like the Spectrum 128, Amstrad CPC, or MSX), and support for loading .P and .O files for Sinclair's ZX80 and ZX81. Improvements even came from other programmers, such as support for .TSX files for MSX computers, contributed by NataliaPC (Natalia Pujol), and the correction of an error in the LCD printing routine, by Rafael Molina (rcmolina).

It was Rafael, who found the TZXDuino's directory rewinding a horrible way to navigate, and whose improvement ideas didn't convince Duncan Edwards, who considered them to be memory-intensive, who set about optimizing the TZXDuino code. He already had an idea in mind that he finally made public in this post on August 20, 2017 (https://www.va-de-retro.com/foros/viewtopic.php?f=63&t=5541&start=110#p109097 This marked the beginning of his open-source project, MaxDuino, in which he attempted to unify the TZXduino and CASduino firmwares. Previously, if you only had one Duino, you had to install the TZXduino firmware to upload files to a Spectrum, and then install the CASduino firmware to upload files to an MSX. Unifying the firmwares eliminated this need, which was a significant advantage. Furthermore, Rafael has continued to optimize and add more features since then. Moreover, all the OLED screen development, patching of various bugs, time and step counters, menu navigation, polarity inversion, block rewinding, logarithmic search, developments on BBC Micro, improvements on Dragon and Tandy CoCo, support for the id15 block, support for Arduino Nano Every, and Tapuino support on TZXDuino for Commodore 64, are many of its improvements that have later been incorporated into the original TZXDuino project.

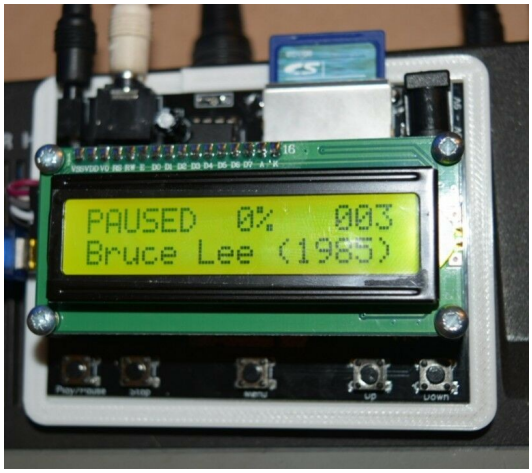
Thanks to MaxDuino's open-source nature and collaborative approach, Rafael Molina has received code contributions from many people, which have helped improve the project. He has also received assistance from other programmers, such as Natalia Pujol (NataliaPC), Alfredo “Retrocant” (@acf76es), Spirax, and Ramón Martínez Palomares.*(ramp),*without forgetting the original projects of Sweetlilme -Tapuino-, and those of Duncan Edwards and Andrew Beer -CASduino and TZXduino-.

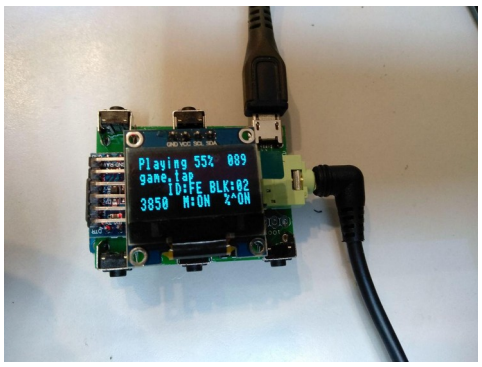
To follow the evolution of this project you can read its official "thread" at va-deretro.com (https://www.va-de-retro.com/foros/viewtopic.php?t=5541&start=9999) and the source code is available on their GitHub:https://github.com/rcmolina/MaxDuino_v1.54 .

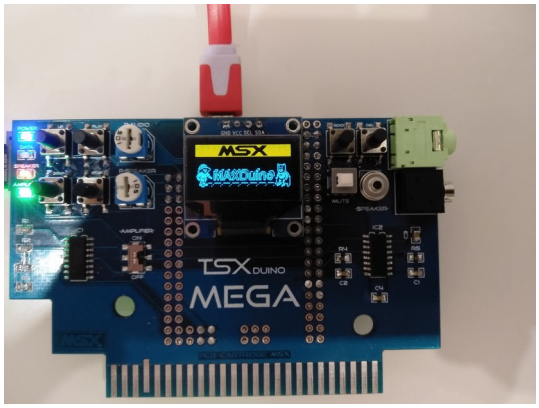
5. What is a MaxDuino device like?

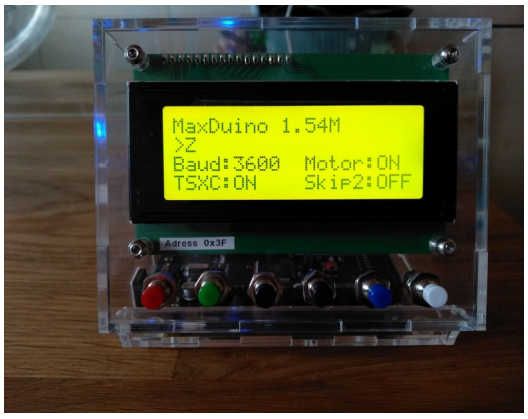
As I mentioned before, the firmware is called MaxDuino, but the devices themselves may have various names such as Tzxduino, Maxduino, Miniduino, or Megaduino. These devices are "homemade," and those who build them generally produce limited runs, which are discontinued once completed. Typically, in these cases, the information is made available to everyone so that each person can build their own, basically the board designs and the component list. However, there are other manufacturers who have their own online stores and sell their own models under their own names, indicating that they are compatible with Rafael Molina's MaxDuino firmware.

Examples of different models:









A Maxduino device consists of:

- A connector provides the 5V power that the device needs. A MicroUSB port is generally used.

- A slot for inserting an SD or microSD card, depending on the model. The card must be formatted in FAT32 or FAT16; models up to 8GB in capacity will work without problems.

- A 3.5mm stereo jack connector for audio output. It only uses the left channel, so stereo-to-stereo or mono-to-mono cables are compatible.

- A 2.5 mm jack connector for the remote function (control of the cassette motor by the computer, for example on Amstrad CPC, MSX, Acorn Electron, or BBC Micro). Not all MaxDuinos have this connector.

- An LED or OLED screen to display menus and file names. The four supported types are:

    - 16x2 LCD screen (2 lines of 16 characters each)
    - 20x4 LCD screen (4 lines of 20 characters each)
    - 128x32 pixel OLED display (4 lines of 16 characters)
    - 128x64 pixel OLED display (8 lines of 16 characters)

    LCD screens typically display larger characters than OLED screens, making them better for those with less advanced eyesight. Both LED and OLED screens offer the same information; however, OLED and 20x4 LCD screens, with their larger number of lines, can distribute text more effectively and create a more visually appealing display.

- Five buttons or pushbuttons:

    - 1.- Play / Pause
    - 2.-Stop
    - 3.- Up
    - 4.- Down
    - 5.- Root or menu or pivot

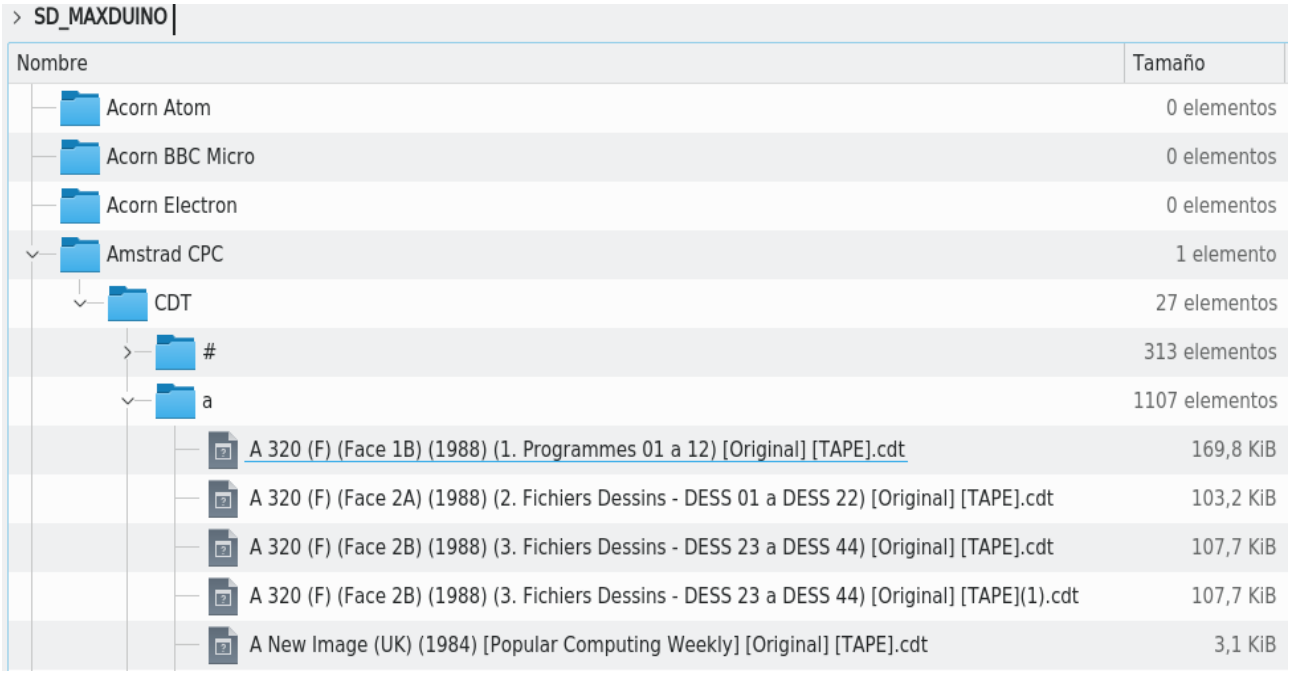
    This button is called three different things because it has had various functions as the firmware has evolved. Its use as a root button is no longer used; this refers to when, in older versions of the TZXDuino firmware (version 1.11 or earlier), it was the only way to navigate back in the directory tree, taking you to the root directory of the SD card. In modern versions of MaxDuino, it can function as a Menu button, displaying the settings menu when pressed, or as a pivot button, allowing access to additional functions when pressed in conjunction with another button.

    Furthermore, the MegaDuino boards have added a 6th button to be used in future improvements such as, for example, the possibility of audio recording.

- An amplifier (for example, a single-channel LM386) is used to boost the sound, which is helpful for systems that are a bit quieter. It can be switched on/off using jumpers on the board. Not all models include an amplifier.

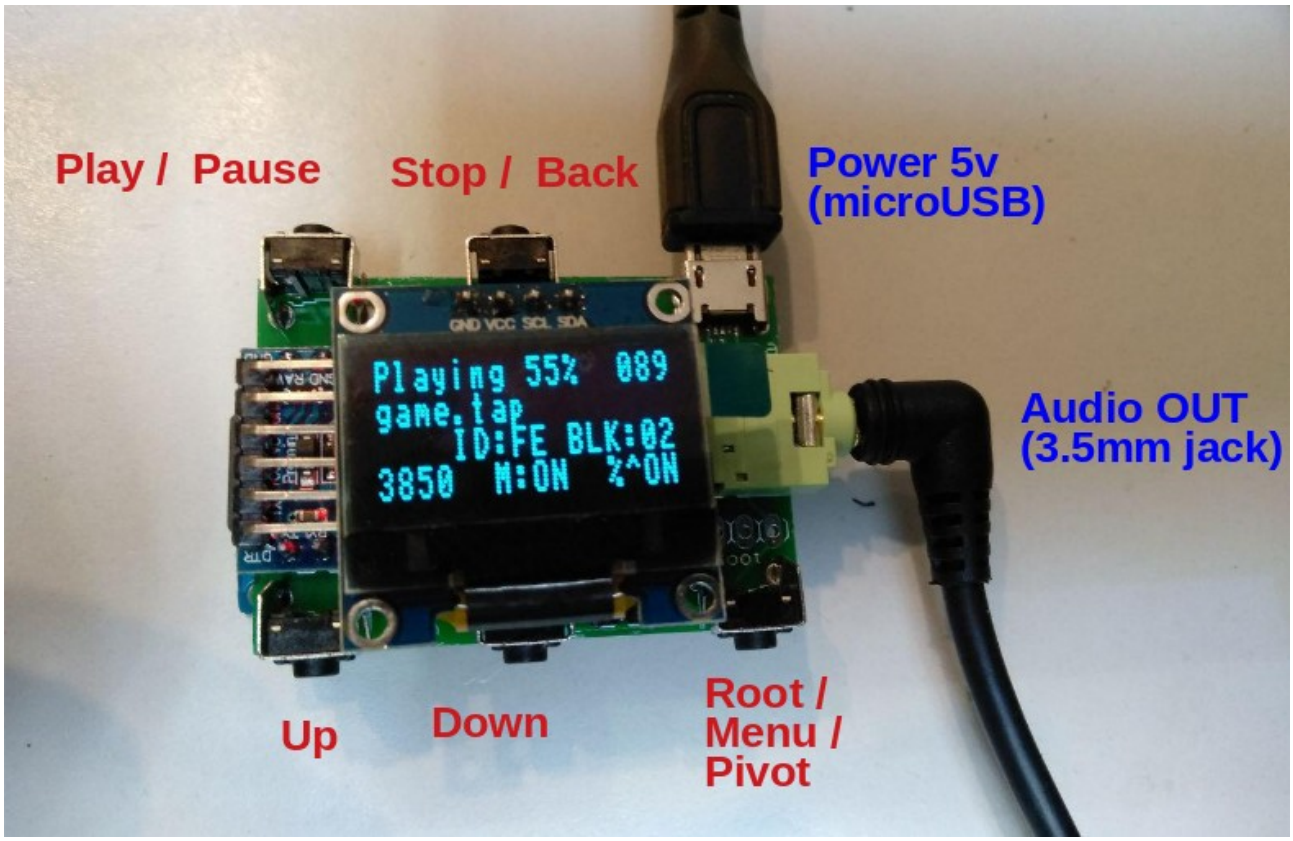
6.-How to operate the MaxDuino?

The first step is to insert an SD or microSD card formatted in FAT32 or FAT16 containing the tape file images. If you have multiple physical machines or an FPGA, a practical approach is to organize the information into folders by system, then by file type (.CAS, .TSX, etc.), and finally separated alphabetically. Having everything organized makes searching easier, but also the later addition of new files.



To explain how it works, I will use my Miniduino, created by Antonio Villena with a casing by Mejias3D.

The arrangement of the 5 buttons on the Miniduino is as follows:



This model has a 1.3-inch OLED screen with a resolution of 128x64 pixels (16 characters in 8 lines). While customizing the firmware might suggest using all 8 lines, it will only utilize the top 3 and the bottom line, leaving 4 lines black in the middle. It's best to customize it by compiling the firmware to use it as a 128x32 display (16 characters in 4 lines). In this configuration, it only uses the even-numbered lines, leaving the odd-numbered lines off, resulting in characters that are 16 pixels high and 8 pixels wide, thus improving readability.

As you can see in the image, this model doesn't have the 2.5mm remote connector, only the 3.5mm audio output jack. This isn't a problem, since some tape file formats have block IDs that indicate when to pause before continuing audio playback.

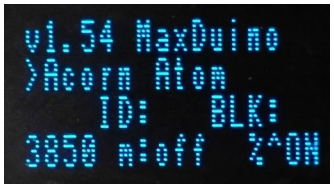
The operation explained below is with firmware version 1.54.

6.1.- Starting the MaxDuino

At startup, a logo may be displayed for a second and a half. When compiling the firmware, you can choose which logo to display from a selection of options.

6.2.- Initial Screen

The first file or directory from the root folder that was saved there will appear. The files and folders are not listed alphabetically, but rather in the chronological order in which they were saved.



- The firmware version is shown in the first row; in this case, it is 1.54.
- The second row shows the folder where we are currently located.
- In the third row, the ID and BLK fields are blank. These fields will be populated when we are positioned within a tape file, and they indicate the current block (BLK) and the block type identifier (ID). Not all block types are displayed to avoid slowing down loading times; only those with IDs 10, 11, 4B, and TAP are shown.

- The fourth row shows the current baud rate for turbo file loading.
  .CAS and .TSX from the MSX platform, whether motor control (remote) is enabled or not (on/off) – useful on platforms that supported remote control such as Amstrad CPC, MSX and BBC Micro
  - , and whether or not the TSXCzxpUEFSW option is enabled, which is used to activate turbo loading (4B blocks) of MSX .CAS and .TSX files, change the polarity of the audio signal in Spectrum and Amstrad CPC files, and parity control in Acorn Electron and BBC Micro .UEF files.

6.3.- Changing the current position

The current position can be changed by pressing the following buttons:

PIVOT: The firmware version disappears from the top line and is replaced by the directory or file we are currently in.

Of the three values in the top row, the middle one is the position of the file or directory we are currently in within the current folder, and the other two are the start and end values of the current range. When we enter a folder, the first value is always the first file or directory, and the third is the last file or directory in that folder.

With 1 < 1 < 17 when entering a directory it is telling us that there are 17 directories or files (3rd value), and that we are positioned in the first one (2nd value).

DOWNTo scroll down within the list of files and directories in a folder, you can press and hold it and it will scroll automatically.*autoscrolling)*.

UPTo move up within the list of files and directories in a folder, we can hold it down and it will scroll automatically.*autoscrolling)*.

PLAY / PAUSE: Being positioned above a directory, we enter it.

If we were positioned above the Amstrad CPC directory and pressed PLAY/PAUSE, we entered it. In this case, we are now positioned in the CDT directory, and the line above shows the parent directory we came from.

Here you can enter the CDT folder by pressing PLAY/PAUSE, and then pressing DOWN until you are positioned in the J folder. If you press PIVOT while on it, you will see its position (11 of 27) within the CDT folder.

STOP / BACK: Go back to the next higher directory.

6.4.- Rapid displacements by semi-intervals

This is useful when we want to navigate through a folder that has many files, and using the DOWN button to reach a specific file that is quite far down the list would take many clicks or a lot of time.

The search by semi-intervals consists of narrowing down by halves and always keeping the midpoint of the upper or lower half.

When I enter the folder “/Amstrad CPC / CDT / a” on my SD card and hold down the PIVOT button, I see the following information

1 < 1 < 1077

That is, I am positioned on the 1st file (second value) of a folder that has 1077 files.

I can now navigate using the DOWN (forward) and UP (backward) buttons within the list of programs that begin with "A". It's possible to hold down either the DOWN or UP button and scroll through the list without having to press it multiple times to move forward or backward.

If after moving I hold down the PIVOT button and see something like

1 < 47 < 1077

I am currently positioned in file or directory number 47 within that folder.

If I know the game I want to play is in position 734, I have the option of holding down the DOWN button until I reach it, or using semi-interval scrolling to get closer.

If I press PIVOT + DOWN at the same time, the interval becomes [47, 1077] and the current position will be more or less in the middle of that interval.

Having done this, if I now hold down the PIVOT button I see

47 < 578 < 1107

I just advanced over 500 positions with a single tap!

Following the same system, I advance in semi-intervals, getting closer to the game I'm looking for.

578 < 843 < 1177

Wow, I went too far!

Now I can also go back in half-intervals by pressing PIVOT + UP simultaneously, the new interval being [578, 843]

578 < 710 < 843

I'm already in position 710, I can now continue advancing to the next semi-interval with PIVOT + DOWN

710 < 777 < 843

Now I go back to the previous half-interval with PIVOT + UP

710 < 743 < 777

When we are close, we have to decide whether we want to continue advancing in semi-intervals or use the UP and DOWN buttons to position ourselves in the file or directory we are looking for.

6.5 - Playing a tape file

If we are viewing a tape file, we will see the following information

The first row shows the file size in bytes, and the second row shows the file name, which, if it does not fit within the characters of a row, will be shifted to show the entire name.

We can perform the following actions by pressing the buttons:

PLAY / PAUSE The first time you press it, the tape file will start playing. The first line will display "Playing," the current playback percentage, and the number of seconds elapsed. Pressing it again will pause playback, and the first line will display "Paused." Pressing it a third time will resume playback from where it left off.

While the file is playing, we can see on the third line how we are progressing through the block number (BLK) and the type of block that is being played at that moment (ID).

If we have turbo mode activated (% ÔN), the loading of these types of blocks in MSX .CAS and .TSX files will be played back at the speed currently configured for turbo blocks (on the screen it is currently 3,850 baud).

If we have the “remote” cable connected to the 2.5mm jack, and the motor control option activated (on), the computer will have control to pause the playback of the file whenever it wants, as well as resume it, that is, it has control of the PLAY / PAUSE button.

While on pause we enter***block mode***and we can use these combinations:

> **UP and DOWN**: Navigating through the blocks of the file. This mode is basically used for multi-load games that, in case of failure, forced the player to repeat a previous stage by reloading the cassette. To do this, once a stage of a game has been loaded, which is generally a specific block, if we have to reload it, while in ***block mode***To navigate back, press the UP button until you reach the block number you want to be on, then press PLAY to load it. The block number and type appear in the top row.
>
> **PIVOT + DOWN and PIVOT + UP**: Move through semi-intervals, as explained in section 6.4, but through blocks.
>
> **PIVOT + STOP / BACK**: Activate or deactivate the TSXCzxpUEFSW option at that moment without needing to access the configuration menu.

STOP / BACK: Audio playback stops and returns to the beginning of the file.

6.6.- Configuration Menu

If we are not playing a file, we can access the settings menu by pressing PIVOT + STOP/BACK. Once inside, we can navigate between the different options using the UP and DOWN buttons.

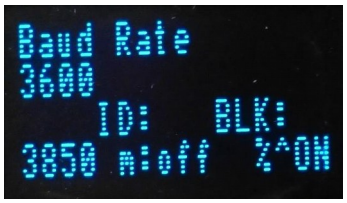
6.6.1.- Baud Rate Option

The first option we can configure is the baud rate. This is the loading speed for turbo blocks in MSX .CAS and .TSX files.

The speeds we can select are: 1,200, 2,400, 3,600 and 3,850 baud.

To access that option, press PLAY.

We'll see 1200 on the second line, and the absence of an asterisk indicates that it's not the current default setting. Pressing DOWN and UP allows us to cycle through the different speeds.



Note that speed 3.850 has an asterisk because it's the current speed for turbo blocks in MSX .CAS and .TSX files. To change to another speed, select it and press PLAY/PAUSE.

The selection of baud rates 1200, 2400, 2700, and 3600 is exclusive to block 4B in MSX .TSX and .CAS formats, and does not affect the loading baud rates of other block types and file types on other platforms.

To exit this option, press STOP / BACK.

6.6.2.- Motor Ctrl Option

The second option that can be configured is the cassette motor control.

If we have the “remote” cable connected to the 2.5mm jack, and the motor control option activated (on), the computer will have control to pause the playback of the file whenever it wants, as well as resume it, that is, it has control of the PLAY / PAUSE button.

Pressing the PLAY/PAUSE button takes you to that option and displays the current value: ON* if it's activated and OFF* if it's deactivated. Pressing PLAY/PAUSE again activates or deactivates the option.

To exit this option, press STOP / BACK.

6.6.3.- TSXCzxpUEFSW option

With this third option we can enable 3 different functionalities, the long name of the option being TSXCONTROLzxpolarityUEFSWITCHPARITY

The features are:

- Enable turbo loading for MSX .CAS and .TSX files
- Changes the polarity of the audio signal in Spectrum and Amstrad CPC files. Necessary for loading certain Amstrad CPC games in CDT format, such as:

  i) No change of polarity: (off) : Tai-Pan, Forbidden Planet, Starbike
  ii) With reversed polarity (on): Basil the Great Mouse Detective, Mask

- Change the parity in the .UEF files of Acorn Electron and BBC Micro. Required for loading games like Starquake or The Sentinel. A list of protected tapes where parity must be enabled can be found at this link:

  http://beebwiki.mdfs.net/List_of_copy_protected_software_titles_on_cassette

Pressing the PLAY/PAUSE button takes you to that option and displays the current value: ON* if it's activated and OFF* if it's deactivated. Pressing PLAY/PAUSE again activates or deactivates the option.

To exit this option, press STOP / BACK.

7.-How do I update the MaxDuino firmware?

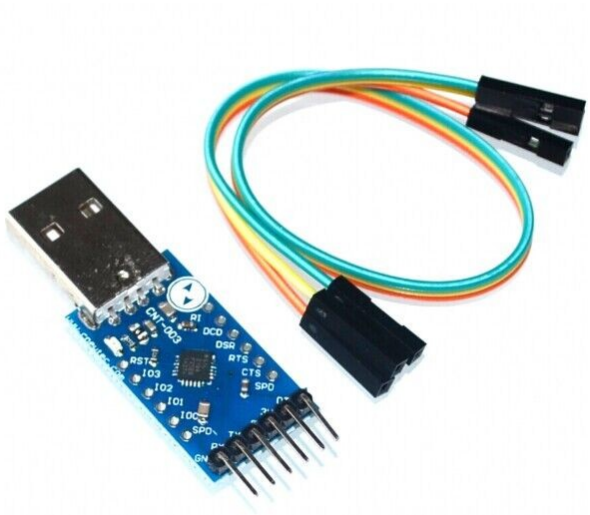
Rafael Molina periodically releases new firmware versions on his GitHub repository to fix bugs and/or add new features. Each time a new version is released, the changes are listed, so it's up to us to decide whether or not to update our board.

I'm going to explain how I do it with my Miniduino, but I suppose it will be quite similar for the other boards.

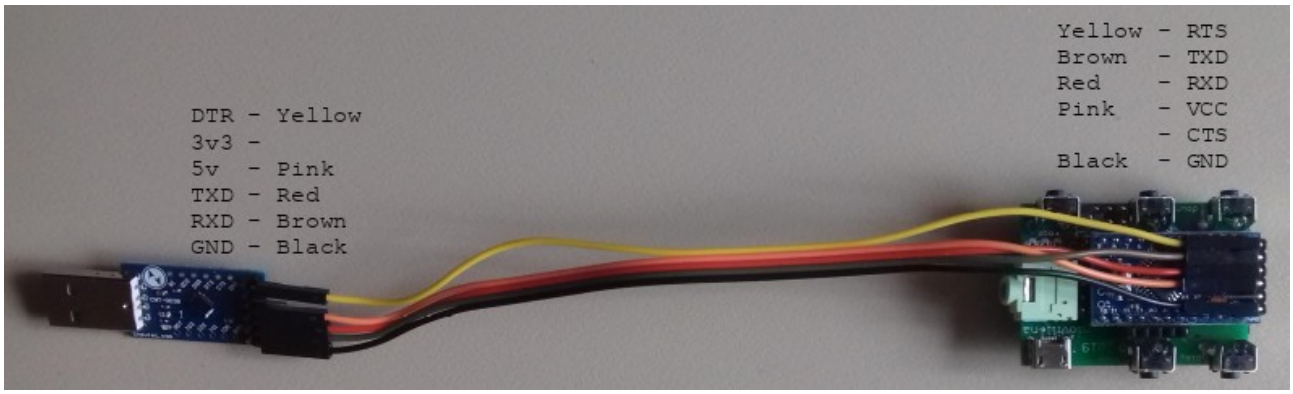
In the case of the Miniduino, the firmware update for the Arduino Nano Pro chip is done through a series of pins on the board. To connect them to the computer, I need a USB-to-serial converter, specifically the one I use in the following:

CP2104 USB to RS232 TTL UART 6PIN Connector Module Serial Converter for Arduino

https://www.ebay.es/itm/CP2104-USB-to-RS232-TTL-UART-6PIN-Connector-Module-Serial-Converter-for-Arduino/183426600814?ssPageName=STRK%3AMEBIDX%3AIT&_trksid=p2057872.m2749.l2649



And the connection between this USB to RS232 converter and the Miniduino board is as follows:

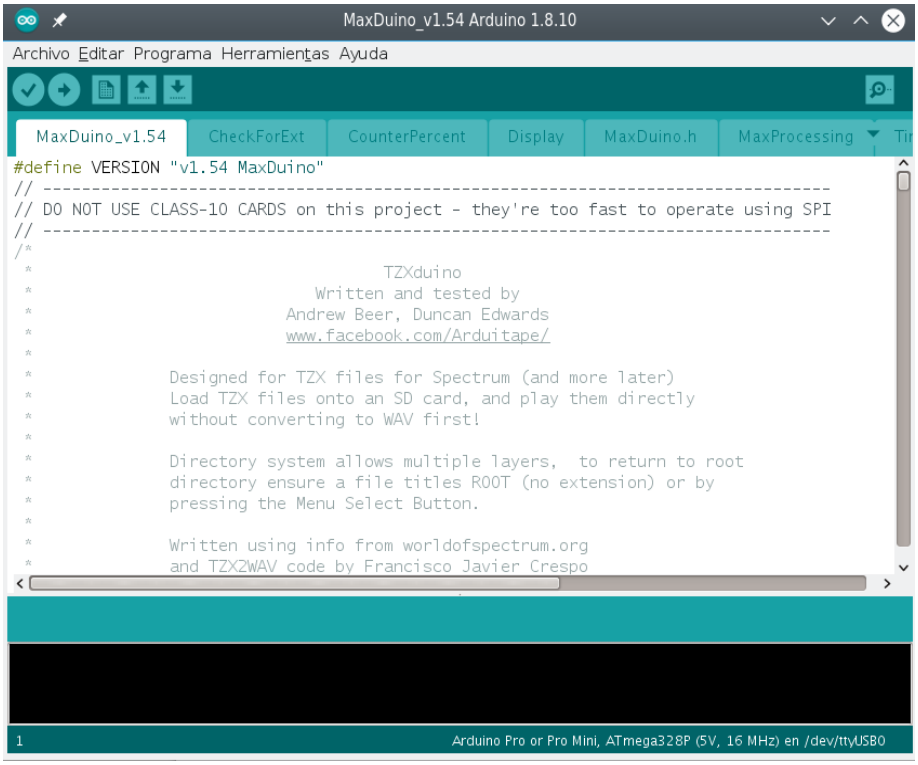


If we don't have the Arduino IDE installed, we access its website (https://www.arduino.cc/en/main/software We download the version for our operating system (Windows, Linux, or Mac) and install it. The version I currently have installed is 1.8.10 for Linux. Rafael Molina recommends version 1.8.4, but I haven't had any problems with the one I've been using.

Once installed, we access Rafael Molina's GitHub (https://github.com/rcmolina) and we download the MaxDuino firmware source code to the computer (the latest version at the moment is 1.54)

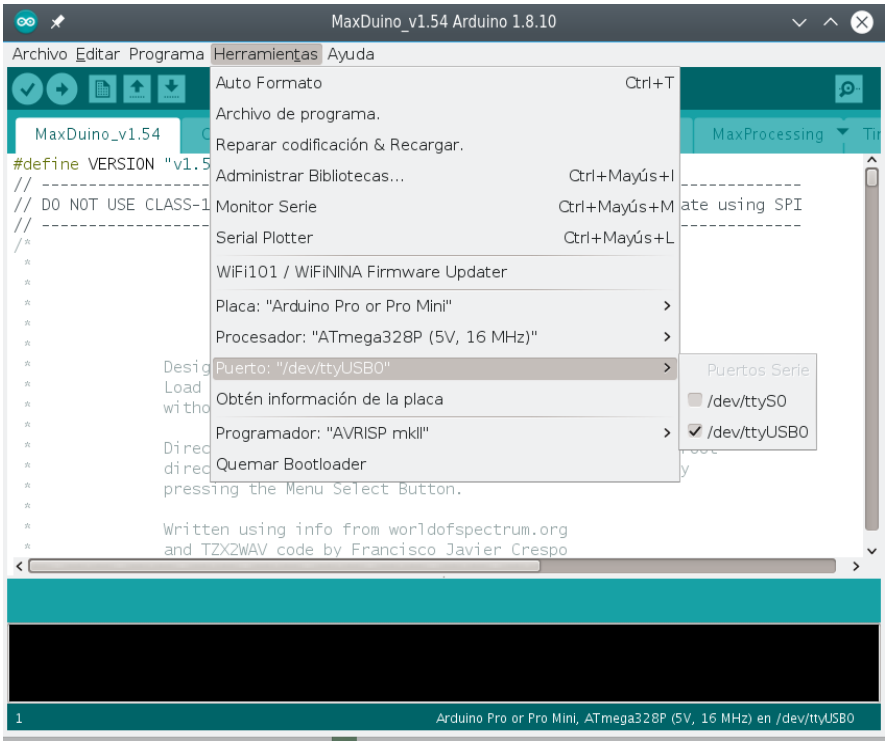
We open the Arduino IDE, go to the folder where we left the sources and load the file MaxDuino_v1.xx.ino (where 1.xx is the version).

We will see that the project code is distributed across a series of tabs:



The next step is to select the board, Arduino type, and port used to communicate with the USB to RS232 converter via the Tools menu. In my case, the options I need to select are:

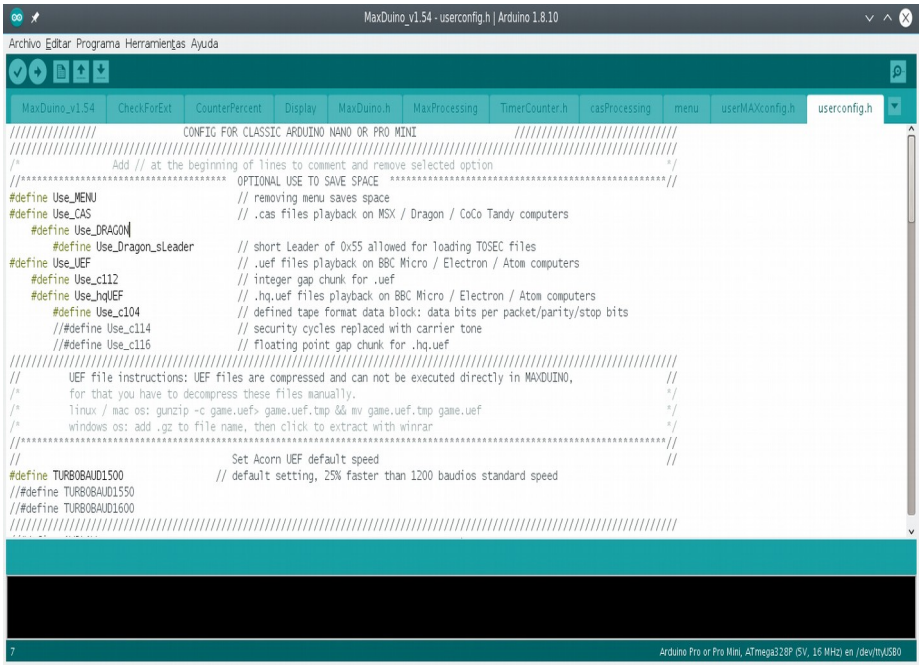
PlateArduino Pro or Pro mini
Processor: ATmega328P (5v, 16Mhz)
Port: */dev/ttyUSB0 (on Windows it's more common to select something like COM4)*



The firmware contains 3 user configuration files, and you must use the appropriate one depending on the processor selected in the IDE:

- userconfig.h → For the Arduino Nano (ATmega328) and the Arduino Pro Mini
- userEVERYconfig.h → For the Arduino Nano Every
- userMAXconfig.h → For the Arduino Mega 2560

Since my Minuino has an Arduino Pro Mini, the file I need to modify is the **userconfig.h**



Definitions are uncommented (remove the double slash //) to take effect, or commented out (add the double slash //) to disable them. Depending on the features we leave enabled, the firmware size will increase. Therefore, if it doesn't fit in the memory of the Arduino chip on our board, we should comment out some definitions to remove them and recompile.

In my case, I've left all the features that are already active by default, and I've only had to customize three things to suit my taste:

- The type of screen used by the Miniduino board
- The PIVOT functionality on the ROOT button.
- The LOGO I want to appear at startup

So the changes I've made are:

 # define OLED1306 // Set if you are using OLED 1306 display // 128x64
// #define OLED1306_128_64 resolution with 8 rows
 # define OLED1106_1_3 // Use this line as well if you have a 1.3" OLED screen

The second line is commented out so that the OLED1306_128_64 definition has no effect. The Miniduino's OLED screen allows for 8 lines, but then the characters appear very small (8x8 size) and only 4 of the 8 rows are usable. Therefore, it's better to leave it commented out, so the information is presented in 4 rows, and the characters are 8x16, making them more legible.

I've also left the following definition uncommented so that the ROOT button functions as a PIVOT

```
# define btnRoot_AS_PIVOT
```

In the end, I didn't make any changes to the LOGO, and with the following definitions, I get the MINI logo that Rafaél created for the Miniduino, but I could have commented on the last definition and selected another logo.

```
# ifdef OLED1306_128_64 //
  #define Maxduino2Alf              // new Maxduino2 logo 128x64 by Alfredo Retrocant //
  # define CosmicCruiser           Dragon32 CosmicCruiser by rcmolina
# else
//#define LOGOARDUITAPE //         // original arduitape logo // new
#define LOGOMAXDUINO               logo created by Spirax
 //#define LOGOMAXDUINO2 //            // new logo2 created by rcmolina //
#define LOGOMAXDUINO3 //             new logo3 created by rcmolina // new
 #define LOGOMAXDUINO4 //             logo4 created by rcmolina // new
#define SUGARLESS_1                logo cretaed bt YoxxxoY // new logo
//#define SUGARLESS_2              created by YoxxxoY
# define LOGOMINIDUINO            // new logo for A.Villena's Miniduino by rcmolina
# endif
```

With the changes made, we would be ready to compile the firmware to our liking and upload it to the Arduino's memory. However, this same file indicates a way to save memory by storing the logo in the EEPROM.

```
//////////////////////////////////////////////////////////////////////////////////////////// // EEPROM LOGO. How to move to EEPROM, saving memory:
// Phase 1: Uncomment RECORD_EEPROM_LOGO define , this copies logo from memory to EEPROM. Compile the sketch.
// Phase 2: Comment RECORD_EEPROM define, uncomment LOAD_EEPROM define. Compile the sketch again

//////////////////////////////////////////////////////////////////////////////////////////// // Also it's possible to select record and load both for better testing new logo activation, pressing MENU simulates a reset.

// And both can be deactivated also showing a black
screen. ////////////////////////////////////////////////////////////////////////////////////////////
```

What you need to do is compile and send the firmware to the board twice:

The first time we left these two lines like this.

```
# define RECORD_EEPROM_LOGO // Uncommenting RECORD_EEPROM deactivates #define Use_MENU // #define LOAD_EEPROM_LOGO
```

Then we click the "Upload" button or press CTRL+U, and if everything is okay, it will compile the firmware and upload it to the board. If there are any errors, they will appear in the lower window of the IDE, and we will have to fix them.

The second time we changed those two lines to:

```
// #define RECORD_EEPROM_LOGO // Uncommenting RECORD_EEPROM deactivates #define Use_MENU
# define LOAD_EEPROM_LOGO
```

And we press the upload button again, which will complete the firmware update.

This action of compiling twice only needs to be done the first time, or if we are going to change the LOGO, for the remaining times we can leave the definitions as the second time.

8. Where can MaxDuino boards be purchased?

They can usually be purchased from online stores of**Antonio Villena**or of**Manuel Fernández Higueras**

- **https://www.antoniovillena.es/store/**
- **https://manuferhi.com/**

Looking on the va-de-retro forum to see if there's a new print run like this one.**merlinkv**

- **MegaDuino print run:https://www.va-de-retro.com/foros/viewtopic.php?f=63&t=8496**

Search on eBay**tzxduino**either**maxduino**

And if you're handy with electronics, you can make it yourself, for example, with the design that Edu Arana shares from his**TZDuino Reloaded:**https://github.com/arananet/TzxDuino-Reloaded

9. Would you like to know more?

The content of this manual is basically an organized presentation of the scattered information found in these threads.**va-de-retro.com**:

- Testing the TZXDuino:https://www.va-de-retro.com/foros/viewtopic.php?f=63&t=5541
- Miniduino (TZXDuino low cost):https://www.va-de-retro.com/foros/viewtopic.php?f=63&t=7448
- Testing the TSXDuino MEGA:https://www.va-de-retro.com/foros/viewtopic.php?f=63&t=8488
- Megaduino print run:https://www.va-de-retro.com/foros/viewtopic.php?f=63&t=8496

and from the project README:

> https://github.com/rcmolina/MaxDuino_v1.54/blob/master/README.md

To learn more about how these tape files are encoded, you can read the document

> **Tutorial for generating TSX files**by Natalia Pujol (NataliaPC) https://github.com/nataliapc/makeTSX/blob/master/docs/Tutorial_TSX_es.pdf

On YouTube we can see many very interesting videos of the MaxDuino in action on Rafa Molina's channel:https://www.youtube.com/channel/UC0L03I8mb4CMytW7MuhF--g/videos

There are also many videos on Alfredo “Retrocant”'s Facebook page: https://www.facebook.com/retrocant

APPENDIX 1. TYPES OF BLOCKS

Format:**TZX**

ID	Worth	Description
ID10	0x10	Standard speed data block
ID11	0x11	Turbo speed data block
ID12	0x12	Pure tone
ID13	0x13	Sequence of pulses of various lengths
ID14	0x14	Pure data block
ID15	0x15	Direct recording block -- TBD - curious to load OTLA files using direct recording (22KHz)
ID18	0x18	CSW recording block**(Not supported)**
ID19	0x19	Generalized data block**(Not supported)**
ID20	0x20	Pause (silence) or 'Stop the tape' command
ID21	0x21	Group start
ID22	0x22	Group end
ID23	0x23	Jump to block
ID24	0x24	Loop start
ID25	0x25	Loop end
ID26	0x26	Call sequence
ID27	0x27	Return from sequence
ID28	0x28	Select block
ID2A	0x2A	Stop the tape if in 48K mode
ID2A	0x2B	Set signal level
ID30	0x30	Text description
ID31	0x31	Message block
ID32	0x32	Archive info
ID33	0x33	Hardware type
ID35	0x35	Custom info block
ID4B	0x4B	Kansas City block (MSX/BBC/Acorn/...)
IDPAUSE	0x59	Custom Pause processing
ID5A	0x5A	Glue block (90 dec, ASCII Letter 'Z')
TUTOR	0xFB	AY file
ZXO	0xFC	ZX80 O file
ZXP	0xFD	ZX81 P File
TAP	0xFE	Tap File Mode
IDEOF	0xFF	End of file

Format:**UEF**

Chunks	Worth	Description
ID0000	0x0000	origin information chunk
ID0100	0x0100	implicit start/stop bit tape data block
ID0104	0x0104	defined tape format data block: data bits per packet/parity/stop bits
ID0110	0x0110	carrier tone (previously high tone)
ID0111	0x0111	carrier tone (previously high tone) with dummy byte at byte
ID0112	0x0112	Integer gap: cycles = (this.baud/1000)*2*n
ID0114	0x0114	Security Cycles replaced with carrier tone
ID0116	0x0116	floating point gap: cycles = floatGap * this.baud
IDCHUNKEOF	0xffff	